EECE 306 Lab 3 notebook. Point Charges, Superposition, and the Field Engine

Team 3: Amelia Harris, Carissa McWilliams, and Mitch Nazareth. TODO replace with your team number and member names.

Fill in every TODO, run `publish('Lab03_notebook.m')` from the toolbox root, print the HTML to PDF, three to five pages.

Contents

- Setup
- Closed form comparison at ten radii
- Radial field direction
- Symmetry on the bisector plane
- Merge functionality
- Dipole far field
- The picture, raw and normalized
- Evaluating the field at the point charge
- Timing at N = 5000
- Field Engine Limits
- Test Output
- Interpretation
- Problems encountered

Setup

```
clear; close all;
rng(306);
e0 = em.const.eps0();
```

Closed form comparison at ten radii

```
Q = 2e-9;
s = em.src.pointCharge(Q, [0 0 0]);
radii = (1:10)';
Enum  = em.field.E(s, [radii zeros(10,2)]);
Eref  = Q ./ (4*pi*e0*radii.^2);
relerr = abs(Enum(:,1) - Eref) ./ Eref;
disp('   r (m)    relative error')
disp([radii relerr])
```

```
   r (m)    relative error
    1.0000         0
    2.0000         0
    3.0000         0
    4.0000         0
    5.0000         0
    6.0000         0
    7.0000         0
    8.0000         0
    9.0000    0.0000
   10.0000         0
```

Radial field direction

E-field is proportional to the radial unit vector rhat, which points away from charge Q. Positive charge = outward; Negative charge = inward.

```
Qpos = em.src.pointCharge(1e-9, [0 0 0]);
Qneg = em.src.pointCharge(-1e-9, [0 0 0]);
pts_dir = randn(50, 3);
Epos = em.field.E(Qpos, pts_dir);
Eneg = em.field.E(Qneg, pts_dir);
```

```
rhat = em.vec.unit(pts_dir);
dot_pos = sum(Epos.* rhat, 2);
dot_neg = sum(Eneg.* rhat, 2);
fprintf(['Minimum charge of positive charges = %.3e (should be ' ...
    'greater than 0)\n'], min(dot_pos));
fprintf(['Maximum charge of negative charges = %.3e (should be less ' ...
    'than 0)\n'], max(dot_neg));
```

```
Minimum charge of positive charges = 7.463e-01 (should be greater than 0)
Maximum charge of negative charges = -7.463e-01 (should be less than 0)
```

Symmetry on the bisector plane

Two equal charges on the x axis. On the plane x = 0 the x component must vanish.

```
q1 = em.src.pointCharge(1e-9, [-0.1 0 0]);
q2 = em.src.pointCharge(1e-9, [ 0.1 0 0]);
s2 = em.src.merge(q1, q2);
pts = [zeros(200,1) randn(200,2)];
Ev  = em.field.E(s2, pts);
tol = 1e-6 * max(em.vec.mag(Ev));
em.test.assertClose(max(abs(Ev(:,1))), 0, tol, 'bisector symmetry');
fprintf('max |Ex| on bisector      = %.3e V/m\n', max(abs(Ev(:,1))));
fprintf('max |E| on the same plane = %.3e V/m\n', max(em.vec.mag(Ev)));
% TODO justify your absolute tolerance for the first number by comparing
% it against the second. State the threshold you chose and why.
%
% The expected value of the x-component of the electric field will cancel
% due to symmetry, so a relative comparison to the observed value would
% give us division by zero. We instead consider the error negligible if the
% difference between the observed and expected values is smaller than an
% absolute tolerance. We choose a small tolerance (10^-6) that scales with
% Ex instead of a fixed number to ensure precision with any magnitude of
% charge.
```

```
max |Ex| on bisector      = 0.000e+00 V/m
max |E| on the same plane = 5.061e+02 V/m
```

Merge functionality

Merge function should concatenate element list for any number of sources

```
sum_count1 = em.src.merge(q1, q2);
fprintf('element count after first merge = %d (expect 2)\n', ...
    size(sum_count1.pos, 1))
% Merge function should return error when a charge source is merged with a
% current source.
crnt_struct = struct('type', 'current', 'pos', [0 0 0], 'I', 1);
try
    try_merge = em.src.merge(q1, crnt_struct);
    fprintf('Mismatch test failed: merge(s1, crnt_struct) failed to raise an error.\n');
catch
    fprintf('Mismatch test passed: merge(s1, crnt_struct) successfully raised an error.\n')
end
```

```
element count after first merge = 2 (expect 2)
Mismatch test passed: merge(s1, crnt_struct) successfully raised an error.
```

Dipole far field

```
sd = em.src.merge(em.src.pointCharge(1e-9,[0 0 0.05]), ...
    em.src.pointCharge(-1e-9,[0 0 -0.05]));
E1 = em.vec.mag(em.field.E(sd, [0 0 10]));
```

```
E2 = em.vec.mag(em.field.E(sd, [0 0 20]));
fprintf('|E(r)| / |E(2r)| = %.4f   (expect near 8 for a dipole)\n', E1/E2);
```

```
|E(r)| / |E(2r)| = 8.0003   (expect near 8 for a dipole)
```

The picture, raw and normalized

```
figure;
em.viz.quiver2(@(r) em.field.E(s2, r), [-0.5 0.5], [-0.5 0.5], 15, ...
    struct('normalize', false, 'title', 'Two equal charges, raw magnitudes'));
figure;
em.viz.quiver2(@(r) em.field.E(s2, r), [-0.5 0.5], [-0.5 0.5], 15, ...
    struct('normalize', true, 'title', 'Two equal charges, direction only'));
% TODO one or two sentences on why the raw plot is unreadable and what
% information the normalized plot gives up in exchange.

% The raw plot is unreadable because the magnitude of the electric field
% lines near the charge is so large that the rest of the lines "disappear".
% Normalizing the plot makes it easier to see the complete field, but the
% intensity of the field lines at each point is lost as a result.
```

Evaluating the field at the point charge

```
try
    Einf = em.field.E(q, [0 0 0]);
    fprintf('Singularity test failed: em.field.E(q, [0 0 0]) failed to raise an error.\n')
catch
    fprintf('Singularity test passed: em.field.E(q, [0 0 0]) successfully raised an error.\n')
end
```

Timing at N = 5000

```
pts5k = randn(5000,3) + 5;
tic; em.field.E(s2, pts5k); t = toc;
fprintf('em.field.E on 5000 points took %.3f s  (requirement, under 2 s)\n', t);
```

```
em.field.E on 5000 points took 0.004 s  (requirement, under 2 s)
```

Field Engine Limits

```
s = em.src.pointCharge(1, [0 0 0]);
sec_max = 5.0;
for N = [1e3 2e3 5e3 1e4 2e4 5e4]
    r = randn(N, 3);
    tic
    Ev = em.field.E(s, r);
    t = toc;
    if t > sec_max
        error('ERROR: Compution time surpasses time threshold at N=%d points: took %.3f seconds\n', N, t)
    end
    fprintf('Observation points: N = %d, Source Charges: M = %d, Product = %d, Time = %.3f s\n', N, size(s.pos, 1), N*size(s.pos, 1), t)
end
```

```
Observation points: N = 1000, Source Charges: M = 1, Product = 1000, Time = 0.001 s
Observation points: N = 2000, Source Charges: M = 1, Product = 2000, Time = 0.001 s
Observation points: N = 5000, Source Charges: M = 1, Product = 5000, Time = 0.001 s
Observation points: N = 10000, Source Charges: M = 1, Product = 10000, Time = 0.001 s
Observation points: N = 20000, Source Charges: M = 1, Product = 20000, Time = 0.003 s
Observation points: N = 50000, Source Charges: M = 1, Product = 50000, Time = 0.008 s
```

Test Output

```
PASS  test_lab03            3.66 s
------------------------------------------------------------
1 passed, 0 failed, 1 total
All tests passing.
```

Interpretation

TODO three to six sentences. Three tests were run above, closed form, symmetry, and a scaling law. Say what each one can catch that the other two cannot.

```
% The closed form test determines if the electric field of a singular point
% charge is precise to a relative tolerance. This test is good at catching
% gross (large) errors in magnitude, but it misses subtle rounding errors.
% The symmetry test determines if the fields of equal and opposite charges
% cancel along the bisector. Using an absolute tolerance that scales with
% the max E-field in that direction catches the subtle errors, but can
% still pass even if the field is the wrong magnitude. The scaling law
% checks if a dipole's far-field magnitude falls off as 1/r^3, using the
% ratio of field at r and 2r. Since this ratio is sensitive to small
% differences and doesn't rely on a closed form or exact symmetry, this
% test is accurate up to a certain limit.
```

Problems encountered

TODO honest account, or NONE. We had to backtrack to Labs 1 and 2 to correct errors in our code. Some of our function returned vectors with incorrect sizes. Despite this, our tests still passed when run, which made the errors easy to miss. We also made a GitHub repository with version control to keep track of our data.

Published with MATLAB® R2024b